Real-Time Object Detection and Tracking

A frame-by-frame performance study of a classical OpenCV segmentation pipeline

Abstract. This paper describes a real-time computer vision pipeline built with OpenCV that segments, detects, counts and tracks moving objects in video using Gaussian filtering, adaptive thresholding, background subtraction, contour analysis and centroid tracking, with live per-class counts drawn on each frame. On a 795-frame, 768x576 pedestrian video running on a laptop CPU, the pipeline needed 11.1 ms per frame on average (90 FPS, about 9 times the source frame rate), with a 95th-percentile latency of 13.5 ms. Background modelling and adaptive thresholding were the most expensive stages. A rough manual check on 9 frames found the detector reported 48 objects where 54 people were present, mainly because touching or occluded people merge into one blob. A one-at-a-time parameter study shows that frame scale is the strongest speed lever and that the unique-object count is highly sensitive to the track-confirmation setting, so that figure should be read as an estimate.

1. Introduction

Detecting and counting moving objects in a video stream is a core computer vision task with uses in traffic monitoring, retail analytics and safety systems. Deep detectors such as YOLO give semantic class labels but need heavy models; classical image processing is much lighter and fully transparent. This work builds the classical end of that spectrum: a pipeline that turns each frame into a segmentation mask, extracts object contours, tracks them across frames and overlays dynamic counts, and then measures, frame by frame, how fast it runs and how it behaves.

The goals are: (i) segment target shapes using a Gaussian filter and an adaptive threshold; (ii) extract contours and keep those inside defined pixel bounds; (iii) overlay live count vectors per object class; (iv) track objects over time; and (v) evaluate per-frame inference metrics and the sensitivity of the results to the main parameters.

2. Method

Each frame passes through seven stages. Every stage is timed individually with a high-resolution clock, so the cost of each step is known for every frame.

Stage	Operation	Purpose / key setting
1. Blur	Grayscale + Gaussian filter	Suppress sensor noise; kernel 5x5
2. Threshold	Adaptive Gaussian threshold (inverse)	Local-contrast mask robust to lighting; block 21, constant 6
3. Background	MOG2 background subtractor	Mask of moving pixels; history 200 frames; shadows removed
4. Morphology	Merge masks, close, open	Fill gaps, remove specks; kernel 5x5
5. Contours	External contours + pixel-bound filter	Keep area >= 150 px^2, shortest side >= 8 px, width/height <= 2.0
6. Tracking	Centroid tracker	Persistent IDs; match radius 60 px; confirm after 5 frames
7. Draw	Boxes, IDs, count panel, FPS	Dynamic overlay on the output frame

Table 1. The seven pipeline stages and their default settings.

2.1 Segmentation

The adaptive threshold alone responds to every textured surface (grass, kerbs, window frames), so it cannot separate objects from background by itself. The pipeline therefore combines it with a background model: the MOG2 foreground mask marks what is moving, the adaptive-threshold mask is kept only inside a dilated copy of that region (so objects are filled with their own contrast edges), and morphological closing and opening join fragments and remove noise. The result is one binary segmentation mask per frame (Figure 1).



Figure 1. Example output (frame 560): annotated frame with boxes, IDs and the live count panel (left) and the final segmentation mask (right).

2.2 Detection, classes and tracking

Contours of the mask are converted to bounding boxes and filtered by pixel bounds: tiny blobs and thin slivers or streaks (for example fluttering barrier tape) are rejected. Remaining blobs are assigned one of three classes by simple shape rules: **person-like** (taller than wide, bounding box up to 6000 px^2), **large blob** (bounding box larger than that, typically several people merged) and **small object** (everything else). These are heuristics, not a trained classifier, and the class names describe shape, not verified identity.

A centroid tracker links detections across frames: each existing track is matched to the nearest new centroid within a radius (closest pairs first), unmatched detections start new tracks, and tracks are dropped after 8 missed frames. A track is only reported once it has persisted for 5 frames, and detections are ignored for the first 20 frames while the background model learns the scene. Both measures remove flicker, at the price of a short delay before an object first appears in the counts.

3. Experimental setup

Input. The OpenCV sample clip *vtest.avi*: a fixed camera watching pedestrians, 768x576 pixels, 795 frames at 10 fps (79.5 s). It has no ground-truth annotations.

Hardware and software. Intel Core i5-1035G1 (4 cores, 8 threads), 7.7 GB RAM, Windows 10, CPU only (no GPU); Python 3.10.5, OpenCV 5.0.0, NumPy 2.2.6.

Metrics. *Pipeline time* is the processing time of one frame through all seven stages, excluding video decoding and encoding; *pipeline FPS* is 1000 divided by the mean pipeline time. *End-to-end FPS* is frames divided by wall-clock time of the whole run including video reading and writing the annotated and mask videos. Timings include the warm-up frames. Timings on a laptop vary between runs: the mean frame time was 10.7, 11.1 and 11.7 ms across three identical full runs, so differences of this size should not be over-interpreted.

4. Results

4.1 Inference performance

Metric	Value
Frames processed	795
Mean pipeline time	11.09 ms (std 2.11 ms)
Median / 95th percentile / max	10.68 / 13.52 / 37.76 ms
Pipeline FPS (1000 / mean time)	90.2
End-to-end FPS (incl. video I/O)	51.5
Real-time margin vs. 10 fps source	9.0x

Table 2. Inference performance over the whole video.

The pipeline processes frames about 9 times faster than the video was recorded, so it would keep up with a live stream on this CPU with a wide margin. Frame times are tight (median 10.7 ms, 95th percentile 13.5 ms) with occasional spikes up to 37.8 ms; the cause of the spikes was not investigated. End-to-end FPS is lower (51) because decoding the video and encoding two output videos are not part of the pipeline time.

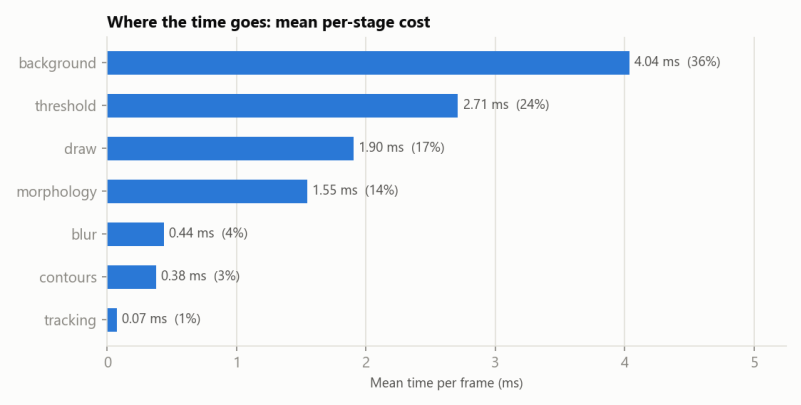
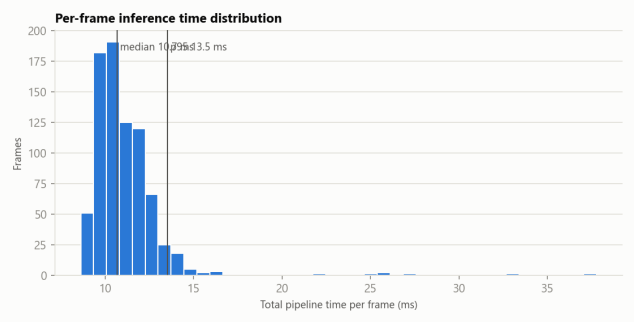


Figure 2. Mean time per stage. Background (36%) and threshold (24%) dominate; tracking costs under 1%.

The stages that touch every pixel dominate: background (4.04 ms), threshold (2.71 ms) and draw (1.90 ms). Contour extraction and tracking together take 0.45 ms, so they are a small part of the total cost in this scene (the effect of much larger object counts was not tested).



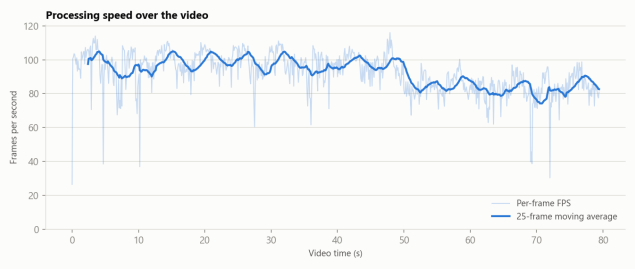


Figure 3 (left). Distribution of per-frame pipeline time with the median and 95th percentile marked. Figure 4 (right). Per-frame FPS (faint) and a 25-frame moving average over the video.

4.2 Frame-by-frame metrics

The complete per-frame record (all stage times, counts and tracking totals for every frame) is saved in *results/frame_metrics.csv*. Table 3 shows nine evenly spaced frames as examples.

Frame	Total ms	FPS	Detections	Person-like	Large blob	Small obj.	Unique so far
80	9.49	105	5	3	1	1	10
160	10.60	94	7	7	0	0	19
240	10.40	96	7	6	0	1	26
320	10.09	99	5	4	0	1	27
400	9.99	100	3	3	0	0	30
480	8.62	116	3	3	0	0	33
560	12.08	83	5	4	1	0	42
640	12.47	80	6	5	1	0	51
720	11.89	84	7	5	1	1	58

Table 3. Frame-by-frame inference metrics at nine evenly spaced frames.

4.3 Detection and counting behaviour

Averaged over the video, the pipeline reports 5.1 objects per frame (maximum 10): 4.0 person-like, 0.7 large blobs and 0.3 small objects. The flat start of the curves is the warm-up and confirmation delay. The cumulative tracker ends at 68 unique objects, which is an upper-bound style estimate: a person who is hidden behind the lamp post for longer than the tracker's memory, or whose blob splits and re-merges, is counted again under a new ID.

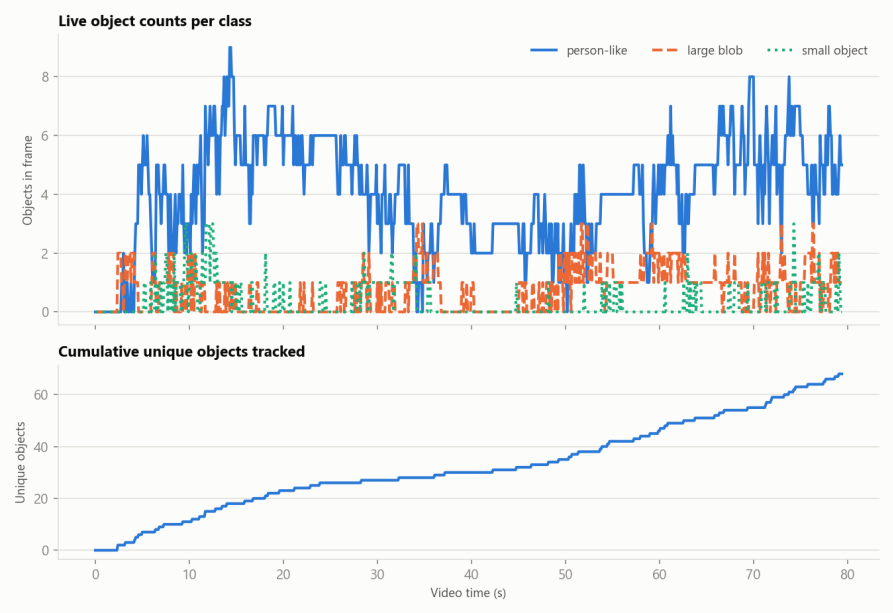


Figure 5. Live counts per class (top) and cumulative unique objects tracked (bottom).

Manual sanity check. The clip has no labels, so people were counted by eye in the raw (un-annotated) frames at 9 evenly spaced positions (every 80th frame) and compared with the detector's output. This is a small, single-observer check, not a rigorous accuracy measurement.

Frame	People counted by eye	Objects detected	Difference
80	7	5	-2
160	7	7	+0
240	7	7	+0
320	6	5	-1
400	3	3	+0
480	4	3	-1
560	6	5	-1
640	6	6	+0
720	8	7	-1
Total	**54**	**48**	**-6**

Table 4. Manual people counts versus detected objects. Mean absolute difference 0.67 objects per frame; exact match in 4 of 9 frames; the detector reported fewer objects than people in 5 frames.

The detector reported 89% as many objects as there were people. Counts are close but not equal, mainly because the detector counts *blobs*: people who walk side by side or overlap merge into one blob, people partly hidden by the lamp post can be split or missed, and very small distant figures may fall below the area threshold. Occasionally a non-person shape is also reported as a small object.

5. Parameter sensitivity

To understand how robust the settings are, one parameter was varied at a time while the others kept their default values, using the first 400 frames (40 s) of the clip. Timing is the faster of two runs. Figure 6 shows mean time per frame, mean detections per frame and the unique-object count; the ringed point marks the default.

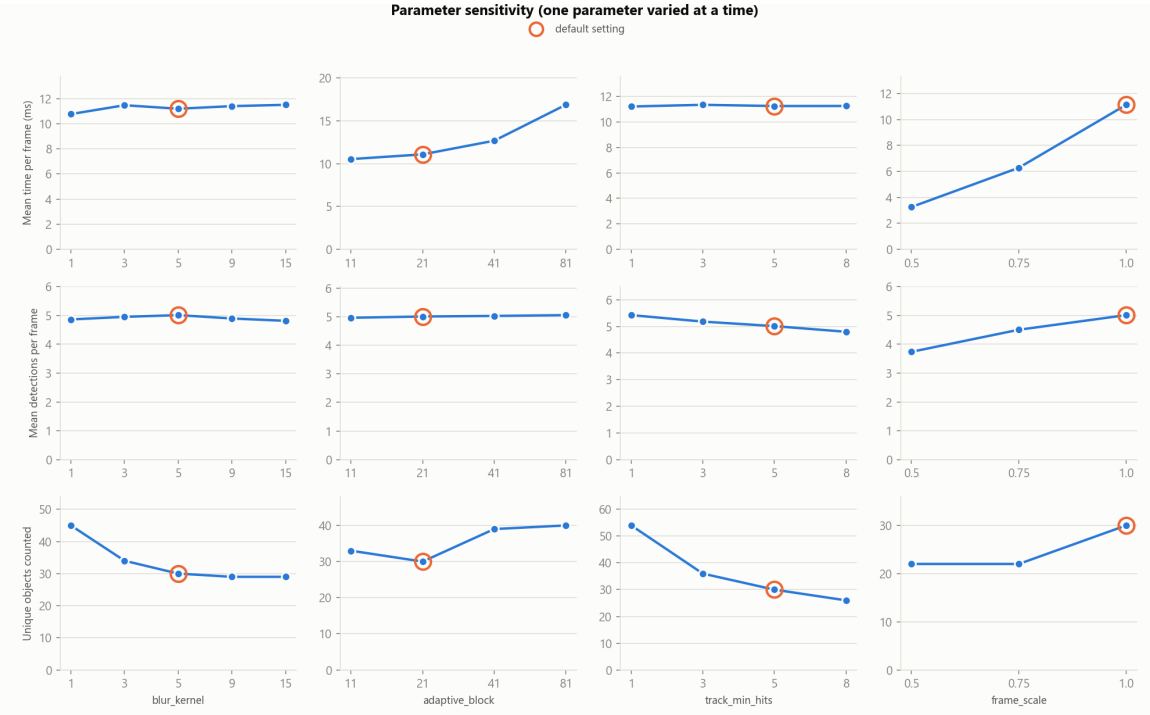


Figure 6. Sensitivity of speed, detections and unique-object count to four parameters.

Frame scale is the strongest speed lever: processing at half resolution takes 3.3 ms per frame against 11.1 ms at full size, but mean detections fall from 5.0 to 3.7 because small, distant people drop below the size thresholds. **Adaptive-threshold block size** mainly affects speed: an 81-pixel window costs 16.9 ms against 11.1 ms for 21, with almost no change in detections per frame. **Blur kernel** hardly changes speed or detections per frame, but stronger smoothing reduces fragmentation and the unique count falls from 45 (no blur) to 29 (15x15). **Track confirmation** has no runtime cost but changes the unique count the most, from 54 (report immediately) to 26 (require 8 frames). The unique-object figure therefore depends strongly on this setting and should not be treated as an exact head count.

6. Discussion and limitations

Strengths. The pipeline is fast, transparent and has no training or model files; every stage can be inspected and timed, and the whole system runs well above real time on a laptop CPU. **Limitations.** (1) Static camera: the background model assumes a fixed viewpoint, so a moving camera or a sudden lighting change would flood the mask with false motion. (2) Only moving objects are detected; people who stand still would be expected to fade into the background model over time (not tested here). (3) Classes are shape heuristics, not semantic labels; a cyclist or a pushed trolley could be labelled person-like. (4) Merged and occluded objects are counted as one blob, and identity can swap or restart after long occlusions because the tracker uses position only. (5) Evaluation is limited to one clip with no ground truth; the manual check covers nine frames and one observer. (6) Timings come from one laptop and vary by roughly 10% between runs.

Possible improvements. Add a pre-trained detector such as YOLO for semantic classes and better separation of touching people; use a Kalman filter or appearance features in the tracker to survive occlusions; split merged blobs with distance-transform or watershed methods; and evaluate on a labelled tracking benchmark to obtain proper precision, recall and tracking-accuracy numbers.

7. Conclusion

A classical OpenCV pipeline of Gaussian filtering, adaptive thresholding, background subtraction, contour extraction and centroid tracking can segment, count and track pedestrians in real time on a laptop CPU, averaging 11.1 ms per frame (90 FPS). Its object counts were close to a manual count on a small sample (48 detected against 54 people), but it cannot separate merged people, and the cumulative unique-object figure depends strongly on tracker settings. The measurements and parameter study identify where time is spent and which settings matter, giving a clear baseline against which a learned detector could later be compared.

8. Reproducibility

All results in this document are generated by the accompanying code. From the project folder: `python download_video.py`, `python main.py` (video, masks, per-frame metrics, charts), `python run_experiments.py` (parameter study) and `python docs/build_whitepaper.py` (this PDF).

Appendix A. Default configuration

Parameter	Default	Meaning
blur_kernel	5	Gaussian blur kernel size (px)
adaptive_block	21	Adaptive-threshold neighbourhood size (px)
adaptive_c	6	Constant subtracted from the local mean
bg_history	200	Frames remembered by the background model
bg_var_threshold	25	MOG2 variance threshold for foreground
morph_kernel	5	Morphology kernel size (px)
min_area	150	Smallest contour area kept (px^2)
max_person_box	6000	Bounding-box area above which a blob is a large blob (px^2)
person_min_aspect	1.1	Minimum height/width for person-like
max_aspect	2	Maximum width/height (thin-streak filter)
min_side	8	Shortest bounding-box side kept (px)
track_max_distance	60	Tracker match radius (px)
track_max_missing	8	Frames a hidden object is remembered
track_min_hits	5	Frames before an object is reported
warmup_frames	20	Initial frames ignored while the background learns

Table A1. All pipeline parameters at their default values (used for every result except the parameter study).

Appendix B. Output files

File	Contents
results/annotated.mp4, mask.mp4	Annotated output video and the per-frame segmentation masks
results/frame_metrics.csv	Per-frame stage times, total time, FPS, detections, class counts, unique objects
results/summary.json	Aggregate metrics and the configuration used
results/experiments.csv	Parameter-study measurements
results/chart_*.png, demo.gif, frames/	Charts, demo animation and sampled frames and masks
docs/manual_counts.csv	Manual people counts and detector output at the nine sampled frames